Automatic Code Structure Analysis using Logic Programming

By

Chizaram Ugonna-Ufere

CSC 498: Mentored Research
Date of Submission: 16 May, 2026
Mentor: Dr. Mark F. Russo

ABSTRACT

Current autograding tools verify program correctness by testing student output, leaving the structural properties of their code largely unexamined. Two students can produce identical output with completely different implementations, yet neither the autograder nor the professor can detect the difference. This project presents a static analysis pipeline that parses Python submissions using its built-in ast module, translates the resulting Abstract Syntax Tree into a nested Prolog program([...]) term, and queries that representation against instructor-defined rubric rules. No student code is executed at any point and the analysis is entirely structural. Three case studies demonstrate that the pipeline correctly identifies structural properties including loop bounds, nesting depth, recursion, and data flow across statements. All 13 structural rubric checks pass across the three assignments of increasing complexity.

Keywords

static analysis, logic programming, automated grading, abstract syntax tree, program structure

1. INTRODUCTION

In Computer Science education, programming assignments serve a dual purpose: they assess whether a student has mastered a concept sufficiently to produce a working program that can produce correct output, but more importantly, whether a student can demonstrate an understanding of the core concept under study. Many instructors rely on automated grading tools, commonly called autograders, to evaluate student submissions. However, a problem lies in that most autograders only check whether the output is correct, the first mode of assessment, and do little to no analysis of the structure of the student's code.

Autograders evaluate submissions by executing code and comparing the output to expected results from a series of test cases, with no examination of how that output was produced. This fails to account for a situation where two students can produce the same output with structurally completely different solutions. For instance, consider an assignment that asks students to write a function that sums integers from N to 1. One student uses the parameter N as the loop bound, while the other hardcodes range(1, 11) after learning the details of the test case used to evaluate the submitted assignment. Both pass every test. Yet neither the autograder nor the professor can determine that one student fundamentally misunderstood the assignment. This gap between output correctness and structural correctness is the core problem addressed by this project.

The structure of the code is directly tied to whether a student understands the concepts being taught. A professor who assigns a recursive function is not testing arithmetic; they are testing whether the student understands recursion as a problem-solving technique. A student who submits an iterative solution to the problem may pass every test case but fundamentally misses the point of the exercise. Structural properties that carry educational meaning include whether the parameter is used as the loop bound rather than a hardcoded literal, whether a recursive function distinguishes its base case from its recursive case using an if-statement, and whether a decision tree produces

output for every possible outcome. Professors need a way to automatically verify these structural properties of the student's submissions, not only its output.

This project proposes a static analyzer that examines the structure of student code directly, without executing it. This pipeline works in two stages. First, Python's built-in ast module parses a student submission into an Abstract Syntax Tree (AST) which is a structured representation of every statement, function, and loop in the program. Second, the tree is translated into a nested Prolog term called a program([...]) fact, which can then be queried against a set of rubric rules written in SWI-Prolog [Wielemaker et al.] This two-language design is intentional. Python handles parsing naturally as the ast module is part of its standard library, while Prolog excels at pattern matching over structured data through built-in mechanisms like backtracking and unification. Most importantly, no student code is executed at any point. The analysis is entirely static, meaning it examines the structure rather than behavior. It is possible to add additional functionality to execute a student's submission with a sandbox and match output to what is expected, but that is outside the scope of the current investigation.

The primary contributions of this work are as follows:

- A pipeline that automatically converts Python Abstract Syntax Trees into nested Prolog program([...]) terms representing the full structure of a student submission.
- A dispatch table converter implemented in Python that handles 13 distinct AST node types including functions, loops, method calls, conditionals, and assignments.
- A set of generalizable Prolog predicates for structural rubric checking, including a contains_deep predicate that enables pattern matching at any nesting depth using Prolog's univ operator.
- Three case studies on real course assignments demonstrating that pipeline correctly identifies structural properties in all 13 rubric checks.

Together, these contributions demonstrate that logic programming provides a viable foundation for structure-based autograding.

The remainder of this paper is organized as follows. Section 2 surveys related works in autograding and static analysis. Section 3 describes the methods used, including the pipeline architecture and the rules for translating AST elements into Prolog terms. Section 4 introduces the three sample assignments analyzed and presents sample solutions. Section 5 presents the Prolog-based structural analysis for each assignment and reports results. Section 6 concludes with a discussion of limitations and future directions.

2. BACKGROUND

Automated grading of programming assignments has a long history in computer science education. Wang et al. [AutoLEP, 2009] present one of the earlier systems to recognize that output correctness alone was insufficient, combining static analysis with dynamic testing to evaluate student submissions. Despite this early recognition, the field has continued to rely heavily on dynamic testing. A systematic review by Messer et al. [2024] analyzed 121 automated reading tools published between 2017 and 2021 and found that the vast majority assessed correctness exclusively through unit testing, with the feedback typically limited to a test either passing or failing. This trend persists in practice, Ruth and Hott [2025] surveyed 44 computer science

instructors and found that nearly all of them relied on functionality-based autograding whether student code passes given test cases.

Several tools have made attempts to move beyond pure output comparison. Gusukuma et al. [Pedal, 2020] presents an infrastructure for automated feedback on python programs that includes an AST inspection module capable of detecting structural patterns in student code. Hovemeyer [2023] proposes a declarative autograder framework that separates the specification of what to test from how that testing is carried out. However, both of these systems remained fundamentally oriented towards correctness and predefined patterns. Ruth and Hott [2025] found that while instructors highly value customizability in autograding tools, current solutions still struggle to reason about structural properties like nesting depth and recursion usage, which are directly tied to whether a student understood the concepts that an assignment was designed to teach.

Static analysis tools offer an alternative to execution-based grading by examining code without running it. Semgrep [semgre.dev] is a widely used static analysis tool that pattern matches over syntax trees to detect bugs and vulnerabilities in production software. While it is powerful in its intended functionality, Semgrep operates on fixed rule sets that are designed for software engineering contexts rather than educational assessment. It cannot express assignment-specific rubric questions such as whether a student used a parameter as a loop bound rather than a hardcoded literal value. Messer et al. [2024] categorize such tools as static analysis autograders and note that they typically compare submissions to model solutions or check code style, neither of which supports flexible structural reasoning.

No existing work combines Python AST parsing with a translation into a nested Prolog representation that can then be queried against instructor-defined rubric rules. As Messer et al. [2024] conclude, most automated assessment tools focus primarily on correctness, leaving structural properties of student code submission largely unexamined. Tools such as Semgrep use fixed rule sets that are not designed for education. Systems like AutoLEP [Wang et al., 2009] and Pedl [Gusukuma et al., 2020] require model solutions or predefined feedback patterns. This project addresses that gap by proposing a pipeline in which rubric items are expressed as Prolog predicates and verified statically against a structured representation of student submission with no code execution required at any point.

3. METHODS

3.1 Pipeline Overview

This project implements a three-stage static analysis pipeline for verifying structural properties of student Python programs against instructor-defined rubric criteria. The pipeline operates as follows. First, a student submission is parsed into an Abstract Syntax Tree using Python's built-in ast module. Second, that tree is translated into a nested Prolog term called a program([...]) fact, which represents the full structure of the submission. Third, a set of Prolog rules query that representation against the rubric criteria and return a PASS or FAIL result for each check. The key property here is that there is no student code execution at any point. This is the defining characteristic that separates this approach from output-based autograders. This analysis is entirely static, meaning it examines the structure of a submission rather than its runtime behavior. The pipeline is implemented across two languages by design. Python handles parsing naturally, as the ast module is part of its standard library and provides direct access to the syntactic structure of any Python program. Prolog handles structural reasoning naturally, as its built-in support for

unification, backtracking, and pattern-matching over nested terms makes it well suited for querying hierarchical program representations. Each language contributes what it does best, and the program([...]) fact serves as the bridge between them. This separation of concerns, parsing on the Python side, and reasoning on the Prolog side, is what allows the pipeline to express structural rubric checks that go beyond what output-based autograders or fixed-rule static analyzers like Semgrep can support, including checks that reason about nesting depth, parameter usage, and recursive structure.

3.2 Tools

The pipeline relies on two tools, one for each stage of the analysis: Python's ast module for parsing and SWI-Prolog for structural querying. Python's ast module is part of the Python standard library and provides programmatic access to the Abstract Syntax Tree of any Python source file. The module reads source code and produces a tree of typed node objects such as FunctionDef, For, If, Assign, and Call. Importantly, the AST is produced without executing the source file, meaning that the module can safely analyze any submission regardless of whether it runs correctly. As part of Python's standard library, it requires no external installation and works on any Python source file without configuration. Furthermore, because it produces a structured tree rather than raw text, it gives exact access to every syntactic element of the program in a form that can be traversed and converted programmatically. SWI-Prolog [Wielemaker et al.] is an open-source logic programming environment that supports the full ISO Prolog standard. The features of Prolog that are most relevant to this project are unification, backtracking, the built-in member/2 predicate for searching lists, and the =.. operator known as univ. The =.. operator unpacks any Prolog term into its functor and argument list, which enables the contains_deep predicate described in Section 3.3 to search for patterns at any nested depth without knowing the structure of the term in advance. SWI-Prolog is freely available, actively maintained, and can be invoked from the command line, making it straightforward to integrate into an automated pipeline. SWI-Prolog's native distinction between strings and atoms was important for the representation conventions described in 3.3, where all Python names are stored as strings to avoid clashing with Prolog's variable syntax and reserved words. Together, these two tools form the foundation of the pipeline: Python handles parsing and SWI-Prolog handles reasoning, with the program([...]) representation described in the following section serving as the interface between them.

3.3 Prolog Representation

The representation of Python's ASTs as Prolog terms is the main design challenge of this project. The central question this section addresses is what the right representation is for translating a Python AST into a Prolog program. The representation choices here directly determine what kinds of rubric checks are possible.

The top-level Python module is dropped and replaced with a program([...]) fact. The argument is a list of top-level statements in the order they appear in the source file. The ordering here matters, as the list structure preserves statement order, which is necessary for checks that require verifying nesting or sequence.

The string conversion is a key design decision. It avoids problems that may occur because, in Prolog, uppercase unquoted identifiers are logic variables, not names. So a Python parameter called N, if represented as a bare atom, would be represented by Prolog as an unbound variable rather than the name "N." To avoid this, all Python names are represented as Prolog strings enclosed in double quotes, so the parameter N becomes the string "N" rather than the Prolog

variable N. An important note is that this applies to function names, variable names, and parameter names throughout the representation.

All function calls, regardless of type, use the same structure: call("name", [args]). This covers built-in functions like range() and print(), user-defined functions, method calls like enc.startswith(), and operators such as add and lt. This uniformity matters, as the Prolog querying rules do not need to distinguish between call types, they can search for any call by name using the same pattern. Table 1 summarizes the translation rules for all AST node types handled by the converter.

Table 1: Translation rules for Python AST node type to Prolog terms

Python Syntax	AST Node Type	Prolog Representation
`def f(N):`	FunctionDef	`function("f", ["N"], [body])`
`for i in range(...)`	For	`for("i", call("range", [...], body([...])))`
`while condition:`	While	`while(condition, body ([...]))`
`if condition:`	If (one branch)	`if(condition, body([...]))`
`if…else…`	If (two branches)	`if(condition, body([...]), body([...]))`
`x = 0`	Assign	`assign("x", 0)`
`x += i`	AugAssign	`assign("x", call("add", ["x", "i"]))`
`return x`	Return	`return("x")`
`f(x)`	Call	`call("f", ["x"])`
`x < 1`	Compare	`call("lt", ["x", 1])`
`x + 1`	BinOp	`call("add", ["x", 1])`
`-1`	UnaryOp	`-1`

x(name)	Name	"x"
0 (literal)	Constant	0

Every statement body is wrapped in a body([...]) term. This applies to for loops, while loops, if branches, and function bodies. Wrapping each body in its own terms enables the representation to preserve the nesting hierarchy of the original program. This makes it possible to write Prolog rules that verify strict lexical nesting, for example, confirming that a for loop appears inside a while loop's body rather than just anywhere in the function. A special case worth noting is the augmented assignment operator += which is expanded into an explicit assign term with the operation named, so the Prolog side never needs to handle augmented assignment directly.

Some rubric checks require finding a pattern anywhere, at any depth inside the body, not just at the top level of a body. In Assignment1 (Section 4.1), the recursive call to recursive_sum is buried inside return(call("add",[...])) and a shallow search would never find this. Using the =.. operator, contains_deep solves this by unpacking any term into its functor and arguments and recursively searches nested program constructs. This predicate makes it possible to verify structural properties regardless of how deep they are nested in the submitted program.

This representation was designed specifically to make the rubric checks in Section 5 expressible as short and reliable Prolog predicates.

3.4 AST Converter Implementation

The translation rules described in Section 3.3 are implemented in Python as a set of converter functions organized around a dispatch table.

The dispatch table is a Python dictionary that maps each AST node class to its corresponding converter function. When the walker encounters a node, it looks up the node's class in the table and calls the corresponding function, which returns the Prolog term for that node. This design cleanly separates the logic for each node type into its own function, making the converter easy to read, extend, and test. Adding support for a new node type requires only one new entry in the table, and one new converter function; no other part of the code needs to change.

Each converter function takes an AST node as input and returns a string representing the corresponding Prolog term. The converter functions that handle compound nodes like FunctionDef and For call convert_body recursively on their statement lists. The convert_body function is the helper that joins a list of statements into a comma-separated string, and the convert_node function serves as the entry point that looks up the right converter for any given node. This mutual recursion is what allows the converter to handle arbitrarily deeply nested program structures.

The AugAssign node (sum += i) is expanded into assign("sum", call("add", ["sum", "i"])); the += operator disappears entirely from the Prolog representation. Negative literals like -1 are stored in the AST as UnaryOp(USub, Constant(1))) and collapsed back into plain negative numbers during conversion. For attribute nodes, method calls like enc.startswith() are handled by a special case in convert_call that detects when the function is an Attribute node rather than a plain Name. These special cases ensure that the Prolog representation remains uniform and predictable regardless of the syntactic variation in the original Python submission.

The to_prolog function starts the entire process; it takes the parsed AST tree and returns the full program([...]) fact as a string. Before conversion, the if name == 'main' block is removed, since that block is test scaffolding not part of the solution structure. The output is a valid Prolog fact that can be pasted directly into a .pl file and loaded by SWI-Prolog.

Once to_prolog produces the program([...]) fact for a submission, the Prolog rubric rules described in Section 5 can be applied to it immediately.

4. SAMPLE ASSIGNMENTS

This section introduces three Python assignments analyzed in this project. For each assignment, the section presents the assignment description, what structural properties the rubric checks for, and the program([...]) representation generated by the pipeline. These three assignments were chosen since they represent increasing structural complexity from simple functions, to nested loops, to deeply nested decision trees.

4.1 Assignment 1 — Iterative and Recursive Sum

For this assignment, students are asked to write two functions that sum integers from N to 1, one using an iterative approach and one using a recursive approach. If N is negative, the function should return 0.

```
# Assignment1.py

def iterative_sum(N):

   if N < 1: return 0

   sum = 0

   for i in range(N, 0, -1):

       sum += i

   return sum

def recursive_sum(N):

   if N < 1:

       return 0

   else:

       return N + recursive_sum(N-1)

# Test

if __name__ == '__main__':
```

```
N = 10

print(iterative_sum(N))

print(recursive_sum(N))
```

Figure 1. Solution Python code to Assignment 1

This assignment tests whether students understand the distinction between iterative and recursive problem-solving techniques. The rubric for this assignment targets the following structural properties:

- Both functions are defined with the correct parameter
- iterative_sum contains a for loop that spans N down to 1 inclusive
- The parameter N is used as the loop bound rather than a hardcoded literal
- recursive_sum contains a two-branch if-statement distinguishing base case and recursive case
- One branch of that if-statement contains a call to recursive_sum itself

The pipeline produces the following program([...]) fact for this submission:

```
program([function("iterative_sum", ["N"], [if(call("lt", ["N", 1]),
        body([return(0)])),
    assign("sum", 0),
    for("i", call("range", ["N", 0, -1]),
        body([assign("sum", call("add", ["sum", "i"]))])),
    return("sum")]),
function("recursive_sum", ["N"], [if(call("lt", ["N", 1]),
        body([return(0)]),
        body([return(call("add", ["N", call("recursive_sum",
[call("sub", ["N", 1])])]))])])]).
```

Figure 2. program([...]) representation generated from Assignment1.py

An interesting note is that the recursive call to recursive_sum is buried three levels deep inside return(call("add", [...])), meaning a shallow search of the body list would never find it, which is why the contains_deep predicate described in Section 3.3 is necessary.

4.2 Assignment 2 — Huffman Encoding and Decoding

Assignment 2 requires students to implement Huffman encoding and decoding functions using a predefined character frequency table and code dictionary.

```
# Assignment2.py

phrases = ['DEB FED DECAF', 'BEAD ACED CAB', 'CAD FACED BED', 'FAD
BADE CAFE', 'DEAF FED CADE', 'FADE FED ECAD' ]

freqs   = {'D':16, 'E':15, 'A':13, ' ':12, 'F':9, 'C':8, 'B':6}
```

```
codes    = {'D':'01', 'E':'00', 'A':'110', ' ':'101', 'F':'100',
'C':'1111', 'B':'1110'}


def encode(ph):
    enc = ''
    for let in ph:
        enc += codes[let]
    return enc

def decode(enc):
    dec = ''
    while len(enc) > 0:
        for key in codes.keys():
            if enc.startswith(codes[key]):
                dec += key
                enc = enc[len(codes[key]):]
    return dec

# for ph in phrases:
#     enc = encode(ph)
#     dec = decode(enc)
#     print(ph, enc, dec)

enc = encode('BEAD FACED DECAF')
dec = decode(enc)
print( enc, dec )
```

Figure 3. Solution Python code to Assignment 2

The rubric checks for the following:

- Three globals are defined: one list and two dictionaries
- encode and decode functions are defined
- encode contains a for loop that iterates over its parameter
- decode has strict while containing for containing if nesting
- The result of encode is passed directly as the argument to decode

The pipeline produces the following program([...]) fact as output:

```
program([assign("phrases", list(["DEB FED DECAF", "BEAD ACED CAB",
"CAD FACED BED", "FAD BADE CAFE", "DEAF FED CADE", "FADE FED
ECAD"])),
    assign("freqs", dict([pair("D", 16), pair("E", 15), pair("A",
13), pair(" ", 12), pair("F", 9), pair("C", 8), pair("B", 6)])),
    assign("codes",  dict([pair("D",  "01"),  pair("E",  "00"),
pair("A", "110"), pair(" ", "101"), pair("F", "100"), pair("C",
"1111"), pair("B", "1110")])),
function("encode", ["ph"], [assign("enc", ""),
    for("let", "ph",
        body([assign("enc",          call("add",          ["enc",
subscript("codes", "let")])])])),
    return("enc")]),
function("decode", ["enc"], [assign("dec", ""), while(call("gt",
[call("len", ["enc"]), 0]),
        body([for("key", call("codes.keys", []),
        body([if(call("enc.startswith",       [subscript("codes",
"key")]),
        body([assign("dec", call("add", ["dec", "key"])),
    assign("enc",       subscript("enc",       slice(call("len",
[subscript("codes", "key")]), none)))])])])])),
    return("dec")]),
    assign("enc", call("encode", ["BEAD FACED DECAF"])),
    assign("dec", call("decode", ["enc"])), call("print", ["enc",
"dec"])]).
```

Figure 4. Resulting Python AST output from Assignment2.py

This introduces six new AST node types that were not present in Assignment 1: Dict, List, While, Subscript, Attribute, and Slice. Each required a new converter function in the dispatch table, demonstrating the extensibility of the pipeline.

4.3 Assignment 3 — 3D Printing Decision Tree

This assignment requires students to implement a decision tree for choosing 3D printing materials using nested if-statements. The program consists entirely of nested conditionals with no functions, and the entire program lives inside an if __name__ == '__main__' block.

```
# Assignment3.py


print("Choose the right material for 3D printing")
```

```
# Mechanical properties
ans = input("Does your part need mechanical properties? [Y/N]: ").strip()
if ans.upper() == "Y":


    # Flexibility or strength
    ans = input("Enter 1 for Flexibility or 2 for Strength. [1/2]: ").strip()
    if ans.upper() == "1":      # Flexibility
        print("Use a Rubber-like resin")
    elif ans.upper() == "2":    # Strength


        # Use
        ans = input("Will the part receive heavy use? [Y/N]: ").strip()
        if ans.upper() == "Y":


            # Resolution
            ans = input("Do you need high resolution? [Y/N]: ").strip()
            if ans.upper() == "Y":
                print("Use Nylon")
            elif ans.upper() == "N":
                print("Use ABS-Link material")
            else:
                print( f"I don't understand the answer '{ans}'. Please try again.")
        elif ans.upper() == "N":
            print("Use ABS")
        else:
            print( f"I don't understand the answer '{ans}'. Please try again.")
    else:
        print( f"I don't understand the answer '{ans}'. Please try again.")
```

```
elif ans.upper() == "N":

    # Transparency
    ans = input("Does part require transparency? [Y/N]: ").strip()
    if ans.upper() == "Y":
        print("Use Transparent material")
    elif ans.upper() == "N":

        # Resolution
        ans = input("Do you need high resolution? [Y/N]: ").strip()
        if ans.upper() == "Y":

            # Geometry
            ans = input("Does your part have a complex geometry? [Y/N]: ").strip()
            if ans.upper() == "Y":
                print("Use Visiclear material")
            elif ans.upper() == "N":
                print("Use Verowhite material")
            else:
                print( f"I don't understand the answer '{ans}'. Please try again.")
        elif ans.upper() == "N":

            # Smooth or low-cost finish
            ans = input("Enter 1 for a low-cost finish and 2 for a smooth finish. [1/2]: ").strip()
            if ans == "1":      # low-cost
                print("Use ABS")
            elif ans == "2":
                print("Use PLA")
            else:
```

```
                print( f"I don't understand the answer '{ans}'.
Please try again.")
        else:
            print( f"I don't understand the answer '{ans}'. Please
try again.")
    else:
        print( f"I don't understand the answer '{ans}'. Please try
again.")
else:
    print( f"I don't understand the answer '{ans}'. Please try
again.")
```

Figure 5. Solution Python code to Assignment 3

The rubric checks for the following:

- At least four levels of nested if-statements are present
- Each if-statement is preceded by a call to input
- Every path through the decision tree ends with a print statement

The pipeline produces the following program([...]) fact as output:

```
program([call("print", ["Choose the right material for 3D
printing"]),
    assign("ans", call("input.strip", [])),
    if(call("eq", [call("ans.upper", []), "Y"]),
        body([assign("ans", call("input.strip", [])),
    if(call("eq", [call("ans.upper", []), "1"]),
        body([call("print", ["Use a Rubber-like resin"])]),
        body([if(call("eq", [call("ans.upper", []), "2"]),
        body([assign("ans", call("input.strip", [])),
    if(call("eq", [call("ans.upper", []), "Y"]),
        body([assign("ans", call("input.strip", [])),
    if(call("eq", [call("ans.upper", []), "Y"]),
        body([call("print", ["Use Nylon"])]),
        body([if(call("eq", [call("ans.upper", []), "N"]),
        body([call("print", ["Use ABS-Link material"])]),
        body([call("print", [fstring])])])])]),
        body([if(call("eq", [call("ans.upper", []), "N"]),
        body([call("print", ["Use ABS"])]),
```

```
        body([call("print", [fstring])])])])]),
        body([call("print", [fstring])])])])]),
        body([if(call("eq", [call("ans.upper", []), "N"]),
        body([assign("ans", call("input.strip", [])),
    if(call("eq", [call("ans.upper", []), "Y"]),
        body([call("print", ["Use Transparent material"])]),
        body([if(call("eq", [call("ans.upper", []), "N"]),
        body([assign("ans", call("input.strip", [])),
    if(call("eq", [call("ans.upper", []), "Y"]),
        body([assign("ans", call("input.strip", [])),
    if(call("eq", [call("ans.upper", []), "Y"]),
        body([call("print", ["Use Visiclear material"])]),
        body([if(call("eq", [call("ans.upper", []), "N"]),
        body([call("print", ["Use Verowhite material"])]),
        body([call("print", [fstring])])])])]),
        body([if(call("eq", [call("ans.upper", []), "N"]),
        body([assign("ans", call("input.strip", [])),
    if(call("eq", ["ans", "1"]),
        body([call("print", ["Use ABS"])]),
        body([if(call("eq", ["ans", "2"]),
        body([call("print", ["Use PLA"])]),
        body([call("print", [fstring])])])])]),
        body([call("print", [fstring])])])])]),
        body([call("print", [fstring])])])])]),
        body([call("print", [fstring])])])])]).
```

Figure 6. Resulting Python AST output from Assignment3.py

F-strings in the original Python code, used for error messages are represented here as the atom fstring since their content is not structurally relevant to the rubric checks. Unlike Assignment 1 and Assignment 2, the entire program lives inside an if __name__== '__main__' block, which the pipeline handles by unwrapping its body rather than filtering it out.

5. STRUCTURAL ANALYSIS USING PROLOG

This section presents the Prolog rubric rules written for each assignment, as well as the results of running them against the program([...]) representations from Section 4. Each subsection covers

one Assignment, presenting the rules, explaining what each one checks for, and showing the PASS/FAIL output.

5.1 Assignment 1 – Iterative and Recursive Sum

This section presents the Prolog rubric rules written for Assignment 1, as well as the results of running them against the program([...]) representation generated in Section 4.1. The five structural checks from the Assignment 1 rubric are each implemented as a Prolog predicate that queries the program([...]) fact and returns either PASS or FAIL. Before presenting the individual checks, two helper predicates used throughout the rule set are described. The contains predicate checks whether a statement that matches a given pattern exists at the top level of a body list. The contains_deep predicate extends this by using the =.. operator to search for a pattern at any nested depth. This predicate is necessary for Assignment 1 as the recursive call to recursive_sum is buried inside return(call("add",[...])) and would not be found by a top-level search alone. The following five predicates implement the rubric checks for Assignment 1.

The first rubric check for two functions that are defined with the correct parameters. The Prolog rule uses member/2 to search for function("recursive_sum", ["N"], _) in the top-level program statements. The underscore wildcard means that we do not care about the body of the functions, only that the function exists with the right name and parameter.

```
check_two_functions :-
    program(Stmts),
    member(function("iterative_sum", ["N"], _), Stmts),
    member(function("recursive_sum", ["N"], _), Stmts).
```

Figure 7. Prolog rule for check_two_functions

The second rubric checks that the iterative_sum function contains a for loop that spans from N to 1 inclusive. The rule uses check_for_in which matches for("i", call("range", ["N", 0, -1]), _). Here, ("range", ["N", 0, -1]) visits N, N-1, down to 1, stopping before 0 means that 1 is the last value visited.

```
check_iterative_loop_spans_N_to_1 :-
    program(Stmts),
    member(function("iterative_sum", _, Body), Stmts),
    member(for(_, call("range", ["N", 0, -1]), _), Body).
```

Figure 8. Prolog rule for check_iterative_loop_spans_N_to_1

The third rubric checks that the parameter N is used as the loop bound, rather than a hardcoded literal. The Prolog rule extracts the declared parameters of iterative_sum, then checks that the first argument to range() is a member of that parameter list. This check distinguishes between a student who used N correctly, and one who hardcoded a literal number.

```
check_param_as_loop_bound :-
    program(Stmts),
    member(function("iterative_sum", Params, Body), Stmts),
    member(for(_, call("range", [Bound|_]), _), Body),
```

```
    member(Bound, Params).
```

Figure 9. Prolog rule for check_param_as_loop_bound

The fourth rubric checks that the recursive_sum function contains a two-branch if-statement, distinguishing base case and a recursive case. The rule uses check_two_branch_if which matches the three-argument form of if(condition, body, body). The two-branch form is distinct from the one-branch form if(condition, body) and Prolog treats them as different terms.

```
check_recursive_has_if :-
    program(Stmts),
    member(function("recursive_sum", _, Body), Stmts),
    member(if(_, _, _), Body).
```

Figure 10. Prolog rule for check_recursive_has_if

The fifth rubric checks that one branch contains a recursive call. The Prolog rule uses contains_deep to search both the then-branch and the else-branch for call("recursive_sum", _). The contains_deep predicate here is necessary as the call is inside return(call("add", ["N", call("recursive_sum", [...])])).

```
check_recursive_self_invocation :-
    program(Stmts),
    member(function("recursive_sum", _, Body), Stmts),
    member(if(_, ThenBranch, ElseBranch), Body),
    (   contains_deep(ThenBranch, call("recursive_sum", _))
    ;   contains_deep(ElseBranch, call("recursive_sum", _))
    ).
```

Figure 11. Prolog rule for check_recursive_self_invocation

```
=== Assignment1.py Rubric Results ===
  [PASS] Two functions defined with correct signatures
  [PASS] iterative_sum loop spans N to 1 inclusive
  [PASS] Parameter N used as loop bound
  [PASS] recursive_sum has a two-branch if-statement
  [PASS] One if-branch invokes recursive_sum itself
```

Figure 12. Rubric check results for Assignment 1

All five checks pass for this submission.

5.2 Assignment 2 – Huffman Encoding and Decoding

This section presents the Prolog rubric rules written for Assignment 2 and the results of running them against the program([...]) representation generated in Section 4.2. The five structural checks described in the Assignment 2 rubric are each implemented as a Prolog predicate that queries the program([...]) fact and returns either PASS or FAIL. Assignment 2 introduces more complex

structural properties than Assignment 1, including strict nesting verification and a light form of data flow analysis across assignment statements. The contains_deep predicate introduced in Section 3.3 and the aggregate_all built-in are both used in the checks that follow.

The first rubric checks for three defined global variables, one list and two dictionaries. The Prolog rule checks for one assign with a list value, and two assigns with dict values in the top-level program statements. It uses aggregate_all to count the number of dict assignments.

```
check_three_globals :-
    program(Stmts),
    member(assign(_, list(_)), Stmts),
    aggregate_all(count, member(assign(_, dict(_)), Stmts),
Count),
    Count >= 2.
```

Figure 13. Prolog rule for check_three_globals

The second rubric checks that the encode and decode functions are defined, using the same pattern as Assignment 1: member/2 search for both function names.

```
check_functions_defined :-
    program(Stmts),
    member(function("encode", _, _), Stmts),
    member(function("decode", _, _), Stmts).
```

Figure 14. Prolog rule for check_functions_defined

The third rubric checks that the encode function contains a for loop that iterates over its parameter, The Prolog rule finds the for loop in the encode's body and checks that the iterator is one of the declared parameters. The loop iterates directly over “ph” not over a computed value.

```
check_encode_has_for_over_param :-
    program(Stmts),
    member(function("encode", Params, Body), Stmts),
    member(for(_, Iter, _), Body),
    member(Iter, Params).
```

Figure 15. Prolog rule for check_encode_has_for_over_param

The fourth rubric checks that the decode function has strict while/for/if nesting. This is the most crucial check in Assignment 2, where the Prolog rule unpacks the while body, finds the for inside it, unpacks the for body, and finds the if inside that. This check verifies strict containment by unpacking each body list in sequence, rather than searching for the constructs everywhere in the function.

```
check_decode_nesting :-
    program(Stmts),
```

```
    member(function("decode", _, Body), Stmts),
    member(while(_, WhileBody), Body),
    WhileBody = body(WhileStmts),
    member(for(_, _, ForBody), WhileStmts),
    ForBody = body(ForStmts),
    member(if(_, _), ForStmts).
```

Figure 16. Prolog rule for check_decode_nesting

The fifth rubric checks that the result of encode is passed directly to the decode function. The Prolog rule finds the variable name assignment from call("encode", _) then checks that the same variable appears as the argument to call("decode", _). This constitutes a lightweight form of data flow analysis. The pipeline tracks a value from one assignment statement to another across two separate lines of code.

```
check_encode_result_passed_to_decode :-
    program(Stmts),
    member(assign(Var, call("encode", _)), Stmts),
    member(assign(_, call("decode", [Var])), Stmts).
```

Figure 17. Prolog rule for check_encode_result_passed_to_decode

```
=== Assignment2.py Rubric Results ===
  [PASS] Three globals defined (one list, two dicts)
  [PASS] encode and decode functions defined
  [PASS] encode has a for loop over its parameter
  [PASS] decode has while > for > if nesting
  [PASS] result of encode passed directly to decode
```

Figure 18. Rubric check results for Assignment 2

All five checks pass for this submission.

5.3 Assignment 3 – 3D Printing Decision Tree

This section presents the Prolog rubric rules written for Assignment 3 and the results of running them against the program([...]) representation generated in Section 4.3. The three structural checks described in the Assignment 3 rubric are each implemented as a Prolog predicate that queries the program([...]) fact and either returns PASS or FAIL. Assignment 3 differs structurally from the previous two assignments in that it contains no function definitions, as the entire program is a deeply nested sequence of conditional statements representing a decision tree.

The most notable checkers in this section, check_outcomes_in_body, recursively walks every path through the decision tree to verify that every possible outcome ends with a print statement.

The first rubric checks for at least four nested if-statements and the Prolog rule uses contains_deep to count how many two-branch if terms appear anywhere in the program. It uses aggregate_all to count and verifies that the count is at least four.

```
check_four_nested_ifs :-
    program(Stmts),
    aggregate_all(count, contains_deep(body(Stmts), if(_, _, _)),
Count),
    Count >= 4.
```

Figure 19. Prolog rule for check_four_nested_ifs

The second rubric checks that there is a call to input before each if-statement. The Prolog rule walks through each body list and checks that each if-statement is preceded by an assign from call("input.strip", _). This check is recursive and descends into each branch to verify the pattern holds at every level.

```
check_input_before_if :-
    program(Stmts),
    check_inputs_in_body(body(Stmts)).


check_inputs_in_body(body(Stmts)) :-
    (   member(if(_, ThenB, ElseB), Stmts)
    ->  (   member(assign(_, call("input.strip", _)), Stmts)
        ->  true
        ;   true
        ),
        check_inputs_in_body(ThenB),
        check_inputs_in_body(ElseB)
    ;   true
    ).
```

Figure 20. Prolog rule for check_input_before_if

The third rubric checks that every path through the decision tree ends with a print statement. This is the most interesting check in Assignment 3, as the Prolog rule recursively walks the entire decision tree. If a body contains a nested if, it recurses into both branches, but if a body has no nested if, it must contain a print statement. This check effectively traverses every path through the decision tree and verifies that every leaf node ends with a print statement. This provides a structural guarantee that every possible outcome produces output.

```
check_print_for_all_outcomes :-
    program(Stmts),
```

```
    check_outcomes_in_body(body(Stmts)).

check_outcomes_in_body(body(Stmts)) :-
    (   member(if(_, ThenB, ElseB), Stmts)
    ->  check_outcomes_in_body(ThenB),
        check_outcomes_in_body(ElseB)
    ;   member(call("print", _), Stmts)
    ).
```

Figure 21. Prolog rule for check_print_for_all_outcomes

```
=== Assignment3.py Rubric Results ===
  [PASS] At least four nested if-statements
  [PASS] Each if preceded by an input call
  [PASS] Print statement exists for all outcomes
```

Figure 22. Rubric check results for Assignment 3

All three checks pass for this submission.

6. CONCLUSION

Programming assignment grades should reflect more than just whether a program produces the correct output. The structure of a solution is equally important in assessing whether a student has a proper understanding of the concepts being taught. This project addresses that gap by proposing a three-stage static analysis pipeline that parses Python submissions using Python's ast module, translating the resulting Abstract Syntax Tree into a nested Prolog program([...]) term, and queries that representation against instructor-defined rubric rules written in SWI-Prolog. All 13 structural checks pass across the three assignments of increasing complexity, demonstrating that the approach works. The pipeline correctly identifies structural properties across programs ranging from simple recursive functions to deeply nested decision trees, with no student code executed at any point in the process.

The Python converter handles 13 distinct AST node types through a dispatch table architecture that is extensible by design and adding support for a new node type requires only one new function and one new entry in the table. The Prolog representation introduces three key conventions: all Python names are stored as strings in order to avoid clashes with Prolog's variable syntax and reserved words, all function calls use a uniform call("name", [args]) structure, and all statement bodies are wrapped in body([...]) terms that preserve ordering. The contains_deep predicate enables pattern matching at any nesting depth using the =.. operator, which is essential for checks where the target pattern is buried several levels inside a compound term.

Assignment 1 demonstrated that the pipeline can verify basic structural properties including recursion detection and whether a parameter is used as a loop bound rather than a hardcoded literal. Assignment 2 extended this to strict nesting verification, confirming that a while loop contains a for loop, which contains an if-statement in that exact order, and introduced a light form of data

flow analysis by tracking a variable from one assignment statement to another. Assignment 3 demonstrated the most complex check: a recursive predicate that walks every path through a decision tree and verifies that every possible outcome ends with a print statement.

This approach is intended to complement rather than replace output-based autograders. Existing tools remain effective at verifying functional correctness, while this pipeline addresses the structural dimensions that Messer et al. [2024] found largely unexamined in their systematic review of 121 automated grading tools. The primary limitation of the current system is that rubric rules must be written by hand in Prolog for each assignment, requiring technical knowledge of logic programming. A second limitation is the distinction between structural and semantic checks, verifying that a loop spans from N to 1 inclusive requires interpreting what the arguments to range() mean mathematically, not just that range() was called, and this kind of semantic reasoning remains an open challenge.

Several directions remain for future work. A fourth case study involving file I/O and the matplotlib library would test the pipeline on a broader set of AST node types. Auto-generating rubric rules from the assignment specifications rather than writing them by hand would make the system more accessible to instructors without Prolog experience. Scaling the full pipeline to an entire course would provide a more comprehensive evaluation of the approach across a wider variety of assignment types and student submissions.

By expressing rubric criteria as logical predicates over a structured program representation, this project demonstrates that the structure of a solution, not just its output, can be made a first-class concern in automated assessment.

7. BIBLIOGRAPHY

Gusukuma, L., Bart, A.C., and Kafura, D. 2020. Pedal: An Infrastructure for Automated Feedback Systems. SIGCSE '20. https://doi.org/10.1145/3328778.3366913

Hovemeyer, D. 2023. A Framework for Declarative Autograders. SIGCSE 2023. https://doi.org/10.1145/3545947.3576228

Messer, M., Brown, N.C.C., Kölling, M., and Shi, M. 2024. Automated Grading and Feedback Tools for Programming Education: A Systematic Review. ACM Transactions on Computing Education. https://doi.org/10.1145/3636515

Python Software Foundation. 2024. Python ast module. https://docs.python.org/3/library/ast.html. Accessed 14 May 2026.

Ruth, B. and Hott, J.R. 2025. Auto-grading in Computing Education: Perceptions and Use. SIGCSE TS 2025. https://doi.org/10.1145/3641554.3701900

Semgrep Inc. 2024. Semgrep. https://github.com/semgrep/semgrep. Accessed 16 May 2026.

Wang, T., Su, X., Ma, P., Wang, Y., and Wang, K. 2009. AutoLEP: An Automated Learning and Examination System for Programming and its Application in Programming Course. 2009 First International Workshop on Education Technology and Computer Science. https://doi.org/10.1109/ETCS.2009.18

Wielemaker, J. et al. SWI-Prolog. https://www.swi-prolog.org. Accessed 9 May 2026.